



DevOps

Dr. John-Paul Ore

*“Ready or not, here comes continuous
deployment.”*

**-- The Top 10 Adages in Continuous
Deployment**

Housekeeping 1/2

- COE Career Fair on Wednesday, Sept 18: You have permission to attend the Career Fair during class time **if you notify me by email before Wednesday's class.**
- ...But Heads Up: Workshop 5 (Wednesday) is challenging and missing the workshop time might it even more difficult.



File System



Home

```
jwore@vclvm179-26: ~  
File Edit View Search Terminal Help  
jwore@vclvm179-26:~$ figlet Remote Desktop + terminal  
Remote Desktop  
+ terminal  
jwore@vclvm179-26:~$ figlet nope  
nope  
jwore@vclvm179-26:~$
```



Housekeeping 2/2 : Workshop Antipatterns

#	Workshop approach...	Instead...
1	Remote Desktop (RDP) into a VM and opening a terminal.	SSH directly into VM (headless).
2	Lots of ansible development targeting the same host VM, leaving the VM in a strange state.	Re-provision a fresh instance (a.k.a. “Turn it off and turn it back on.”). Good practice for testing before submission.
3	Staying stuck on one step for +3-6 hours.	EdStem, email the TA, and / or go around the step, trying to make forward progress.
4	Adding SSH tokens to version control.	Don’t ever check in tokens. Exception: DB password for WS-4 & 5.

Today's Learning Objectives

- Overview of Feature Flags.
- Deployment is not Releasing
- What is “Dark Launching” and how do Feature Flags enable it?
- Learn how *software features* become *experiments*.
- CI/CD pipelines from a “Top 10 Adages...” perspective.

Article: *The Top 10 Adages in Continuous Deployment*

(authors: Microsoft, SAS, Red Hat, Mozilla, Cisco IBM, Oracle Lexis-Nexus, NC State)

- 1. Every Feature is an Experiment**
- 2. The Cost of Change is Dead**
- 3. Be Fast to Deploy but Slow (or Slower) to Release**
4. Invest for Survival
5. You are the Support Person
6. Configuration is Code
7. Comfort the Customer with Discomfort
8. Looking Back to Move Forward
9. Invite Privacy and Security In
10. Read or Not, Here it Comes

The Cost of Change is Dead*

(“Top 10 Adages”)

** In continuously deployed (CD) systems*

Non CI/CD: RELATIVE COST TO FIX AN ERROR

TIME ↓	ERROR FOUND DURING....	COST RATIO
	Requirements	1
	Design	3-6
	Coding/Implementation	10
	Development testing (unit)	15-40
	Acceptance testing	30-70
	Operation (field)	40-100

In CI/CD, this is now less true

HOWEVER:
Cost relationships still applies to many real-world systems: Airplanes, power plants, aircraft carriers, etc.

Source:

Barry Boehm's *Software Engineering Economics*, 1981

Data based on 63 plan-driven projects

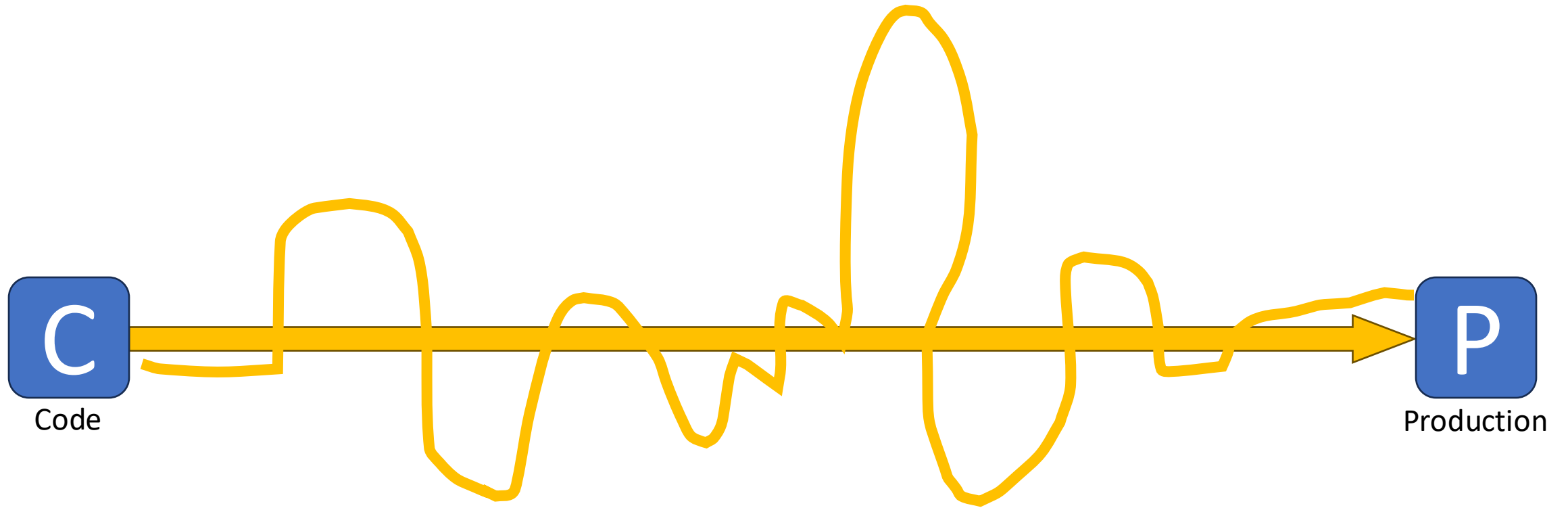


Barry Boehm
(1936-2022)

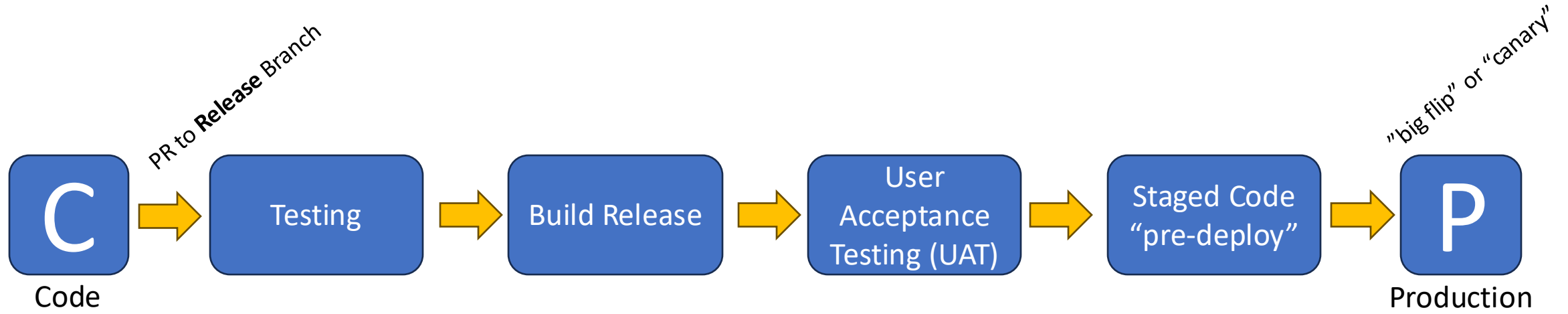
Fast Deployment, Slow(er) Release

(“Top 10 Adages”) feature flags and release engineering

DevOps Pipeline



An example DevOps CD pipeline



Continuous-Deployment Glossary #1

Continuous Deployment	A process in which incremental software changes are automatically tested, vetted, and deployed to production environments.
Feature Flags	A mechanism for dynamically enabling features during production, often controlled by a global in-memory store and cached locally in service instances.
Dark Launching	A practice in which code is incrementally deployed into production but remains invisible to users.
Deflighting or Rollback	A method for rolling back or decommissioning a defective change and removing it from the deployment pipeline.
“Staging” or “Baking”	A stage in the deployment pipeline in which developers test a new software version in a production-like environment. For example, some companies might bake a new version for eight hours before deployment.

Feature Flags

Binary in nature
In control of the developers

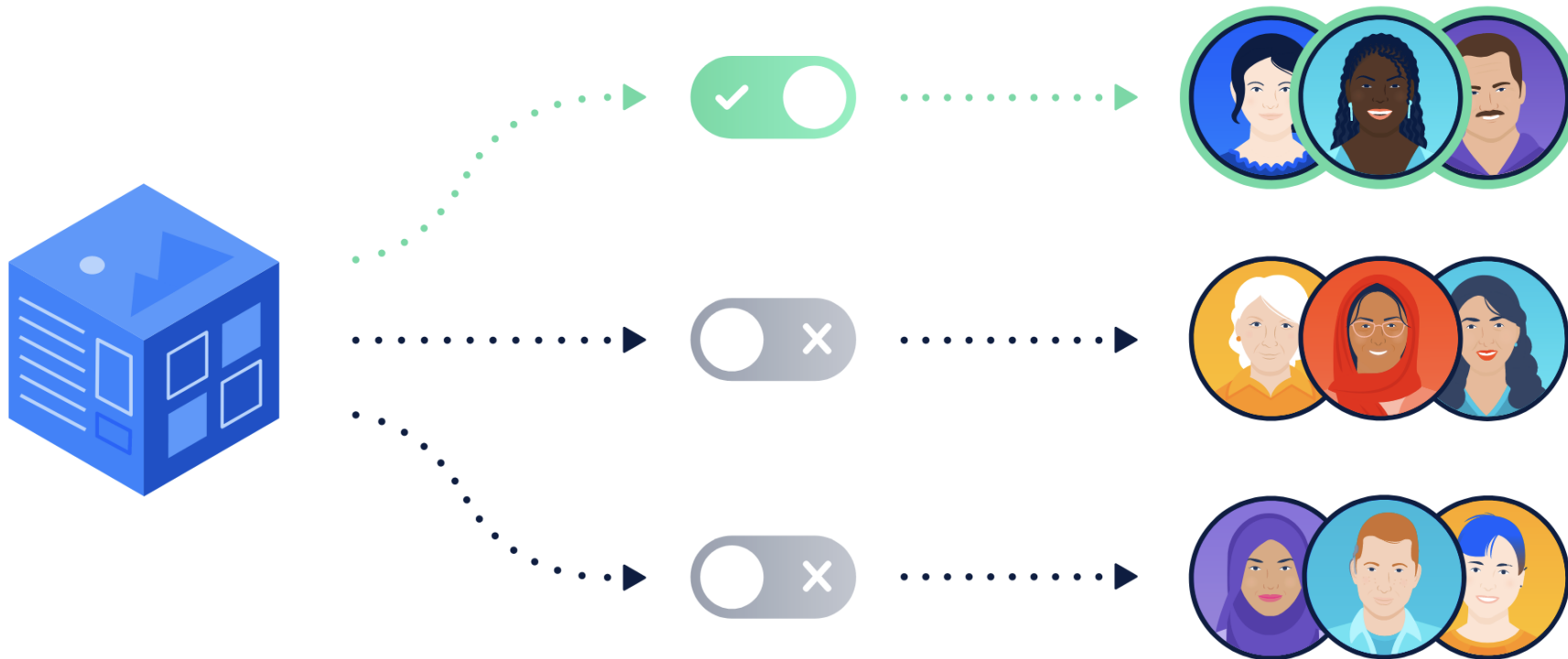
(aka Toggles, Release Toggles, Feature Flippers)

How deployed features become released features

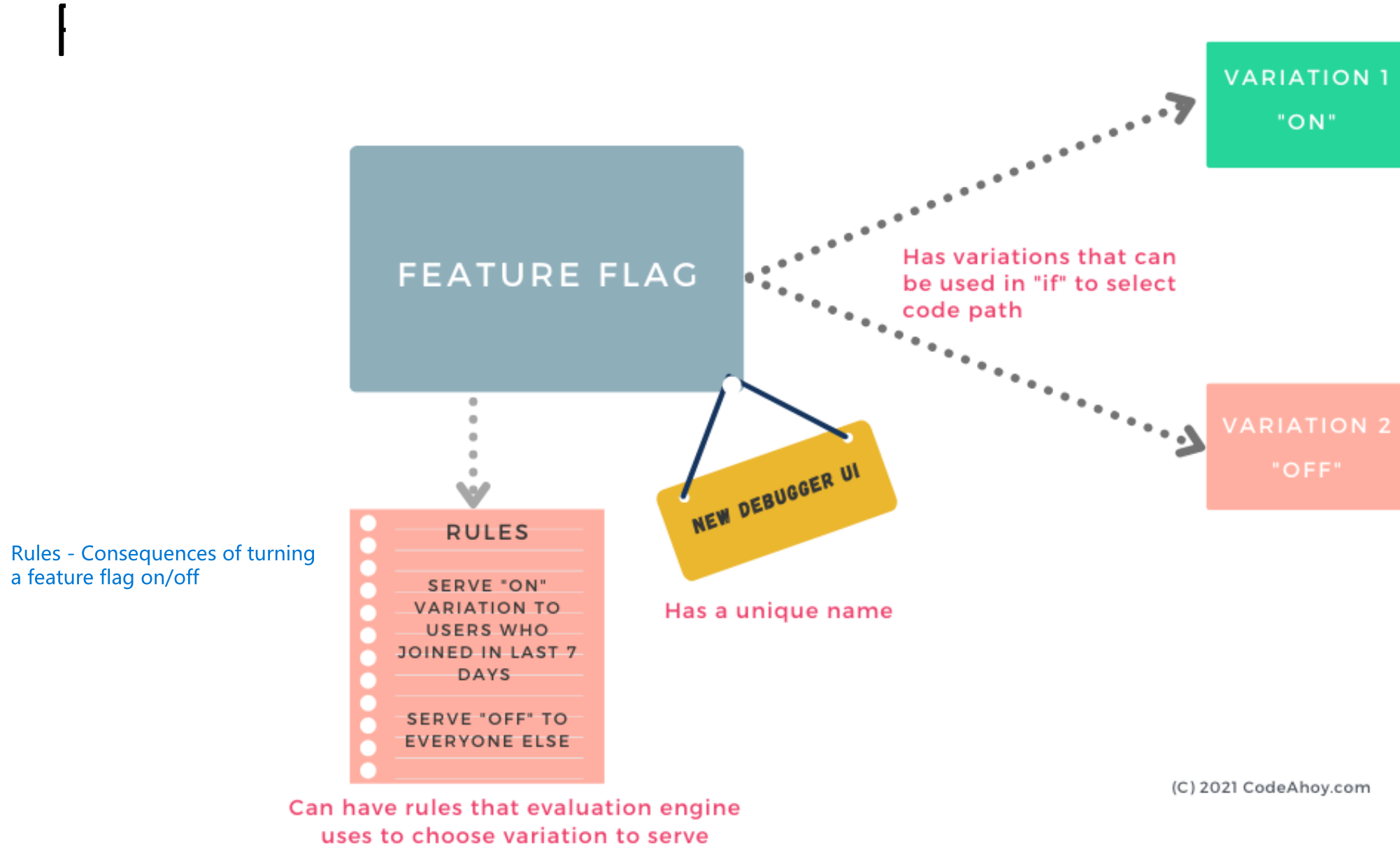
New feature

Feature Flags

Customers

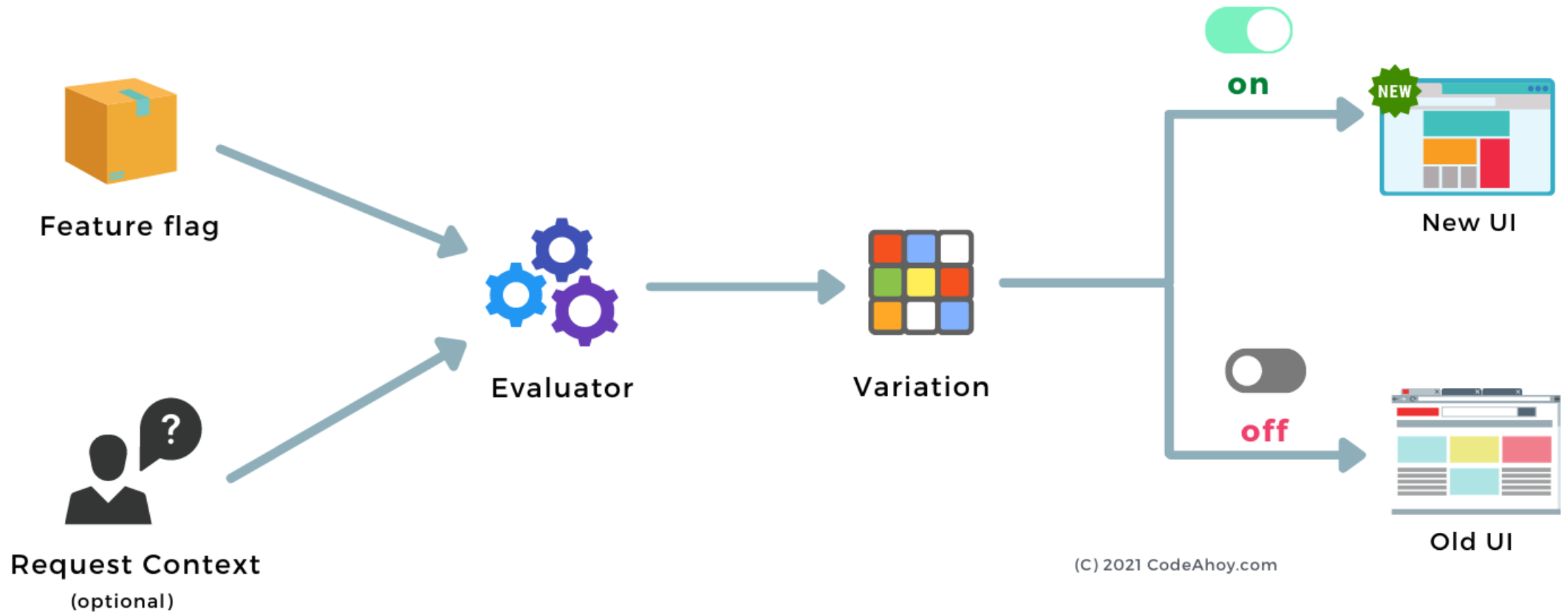


ANATOMY OF A FEATURE FLAG



(C) 2021 CodeAhoy.com

Product “Variation”



Activity

Think-pair-share

You're using an app and the developers of that app are using **Feature Flags** to enable/disable features at runtime.

What could go wrong from a user's perspective?

Unkown state

Violates the trust of the user

Stale Feature Flags → Technical Debt

- *“sub-optimal flag management that makes information on staleness of flag to be ambiguous”*
- *“lack of developer incentives to cleanup source code related to stale flags.”*

2020 IEEE/ACM 42nd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)

PIRANHA: Reducing Feature Flag Debt at Uber

Murali Krishna Ramanathan
Uber Technologies, Inc.
New York City, NY, USA
murali@uber.com

Lazaro Clapp, Rajkishore Barik
Uber Technologies, Inc.
San Francisco, CA, USA
lazaro,rajbarik@uber.com

Manu Sridharan
University of California, Riverside
Riverside, CA, USA
manu@cs.ucr.edu

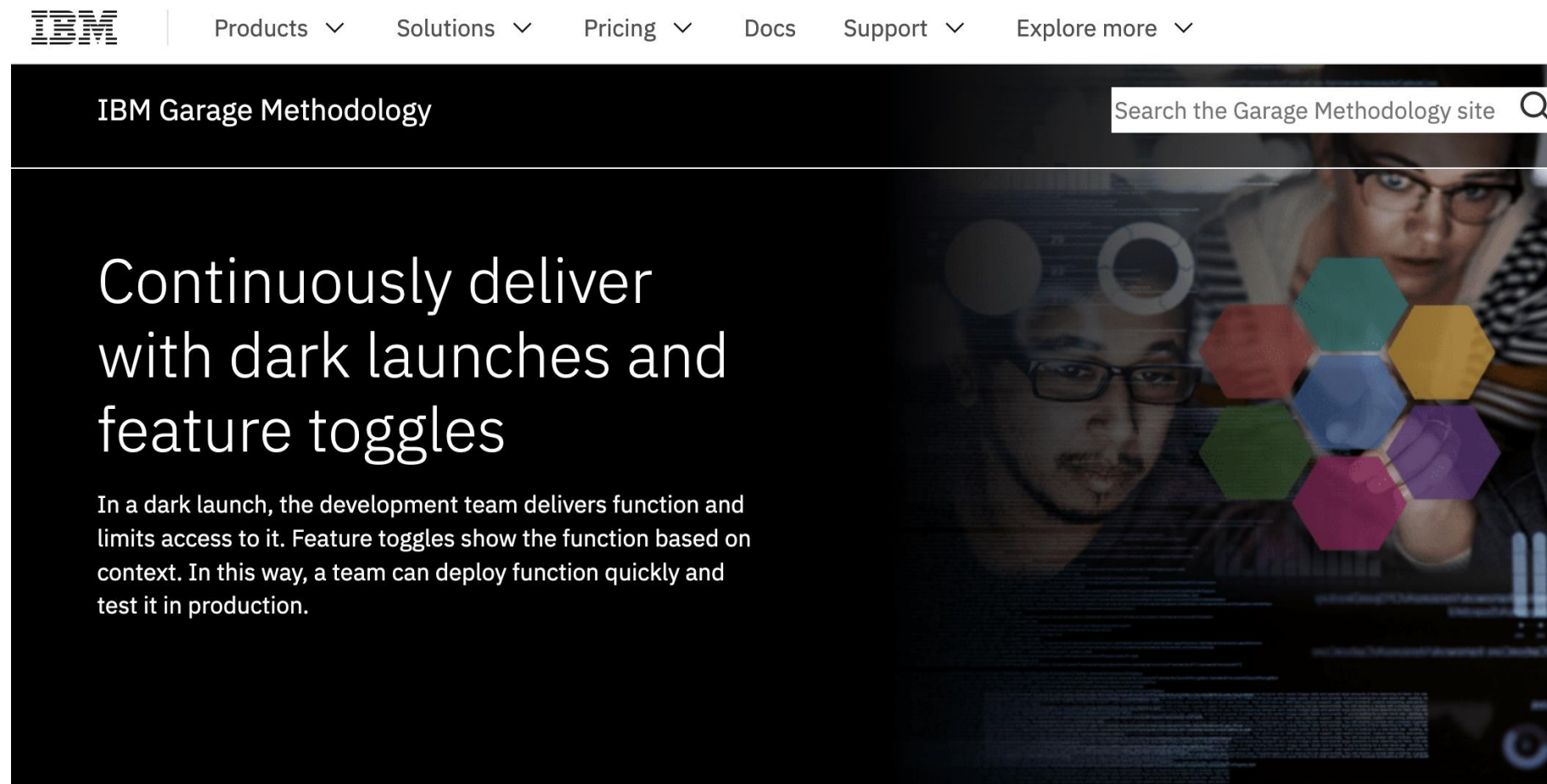
ABSTRACT

Feature flags are commonly used in mobile app development and can introduce technical debt related to deleting their usage from the codebase. This can adversely affect the overall reliability of

1 INTRODUCTION

In traditional software development workflows [1], shipping new features involves determining the set of features to be released in a version, and implementing these features as part of a new branch of

“Dark Launching”



The image is a screenshot of the IBM Garage Methodology website. At the top, there is a navigation bar with the IBM logo on the left and a series of links: Products, Solutions, Pricing, Docs, Support, and Explore more, each followed by a downward arrow. Below the navigation bar is a dark header section. On the left side of this header, the text "IBM Garage Methodology" is displayed. On the right side, there is a search bar with the placeholder text "Search the Garage Methodology site" and a magnifying glass icon. The main body of the page features a large, dark hero section. On the left side of this section, the text "Continuously deliver with dark launches and feature toggles" is written in a large, white, sans-serif font. Below this, a smaller paragraph explains the concept: "In a dark launch, the development team delivers function and limits access to it. Feature toggles show the function based on context. In this way, a team can deploy function quickly and test it in production." On the right side of the hero section, there is a background image of two people, a man and a woman, both wearing glasses and looking at a screen. Overlaid on this image is a cluster of colorful hexagons in shades of teal, yellow, blue, green, and purple, arranged in a circular pattern. The overall aesthetic is modern and tech-oriented.

IBM | Products ▾ Solutions ▾ Pricing ▾ Docs Support ▾ Explore more ▾

IBM Garage Methodology

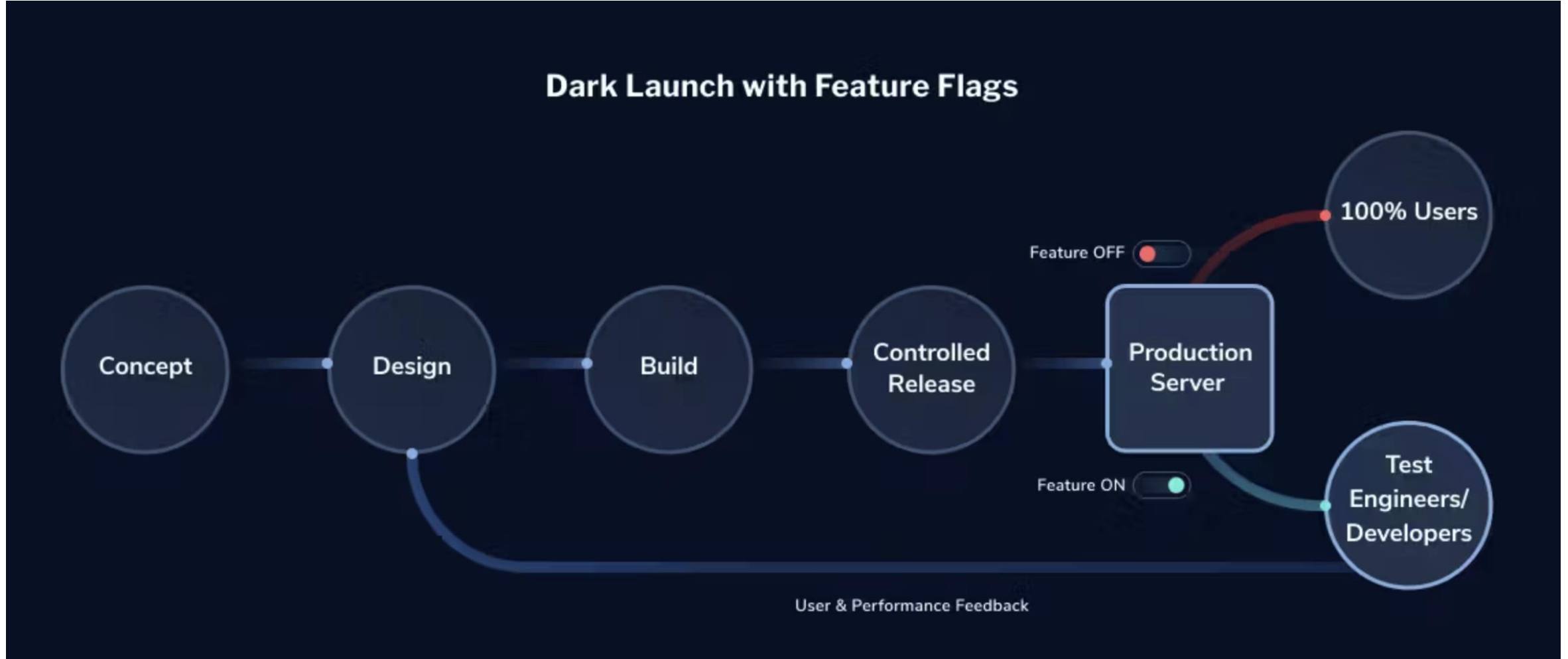
Search the Garage Methodology site 🔍

Continuously deliver with dark launches and feature toggles

In a dark launch, the development team delivers function and limits access to it. Feature toggles show the function based on context. In this way, a team can deploy function quickly and test it in production.

Dark Launch Release for Beta Testing

Released for testing in real environment



Dark Launching

- Enabled by Feature Flags
- “Dark launching can also enable parallel running of a re-implemented feature. The old and new code can both be called and their results checked to see if there are changes with the new algorithm, but only one answer returned to the interface.” – [Martin Fowler](#)

```
{  
    'new.feature': true  
    'some.other.feature': false  
}
```

Dark Launching

Possible use scenarios:

1. Release experimental one-click checkout to see if it increases sales.
2. Test new application infrastructure without switching all traffic.
3. Rollout to VIP users first.
4. Release a feature for beta testing.

```
if (features['new.feature']) {  
    RenderNewFeature()  
} else {  
    RenderOldFeature()  
}
```

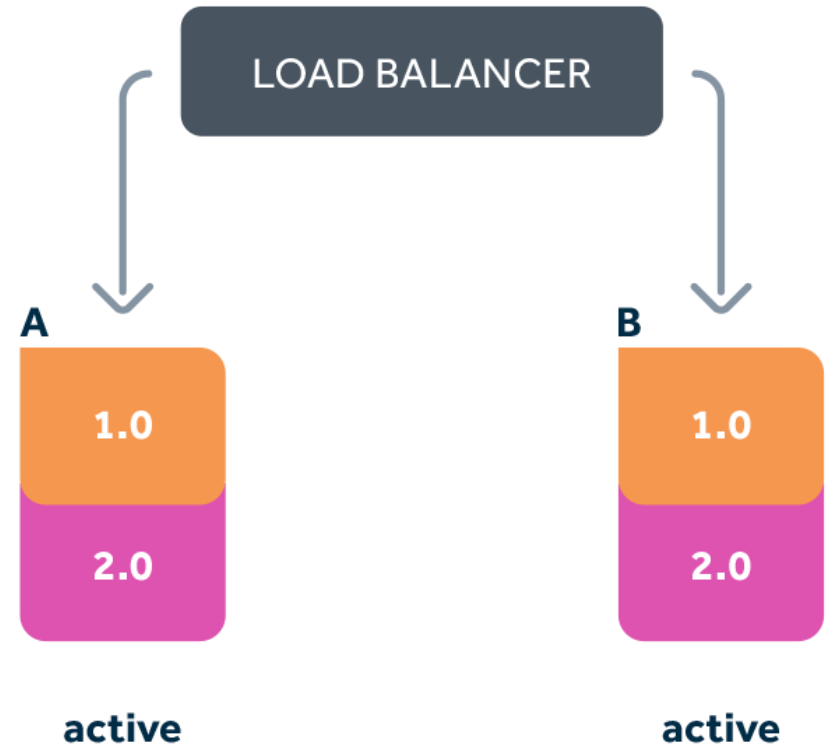
De-Flighting or Rollback

Options:

1. Use load balancer to shift traffic away from Canary Release(s)
2. Disable feature flag (application level, requires push update or equivalent)

In any case, have a rollback plan.

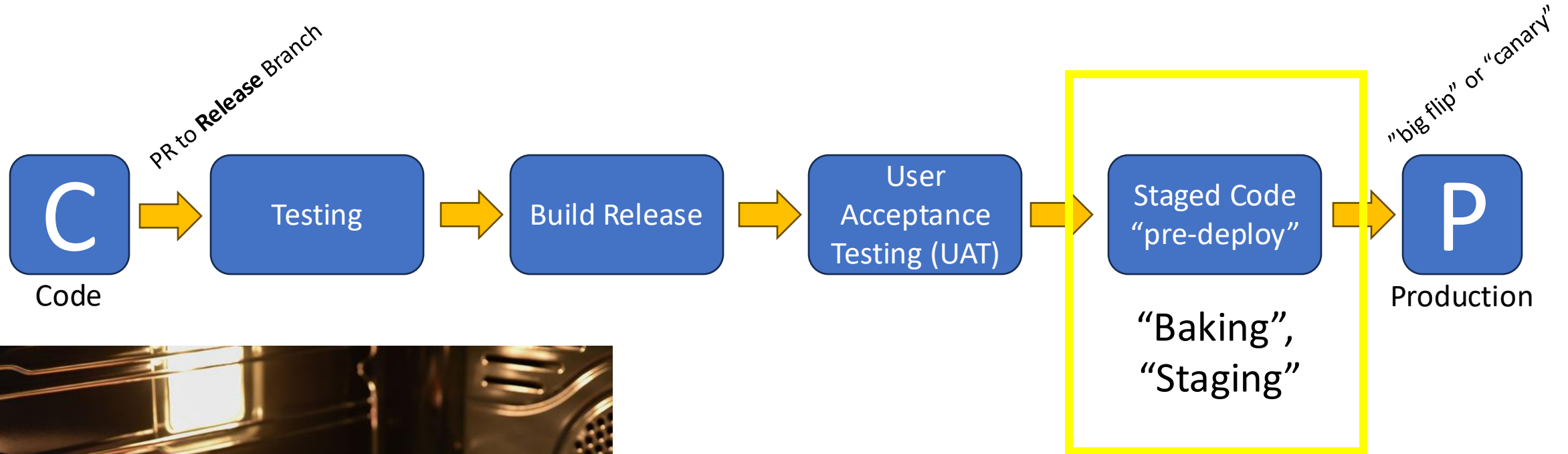
Rollback for a Dark Launch or Feature Toggle



Rollback

If the new feature doesn't behave as desired, send a signal to the application to disable the feature.

An example DevOps CD pipeline



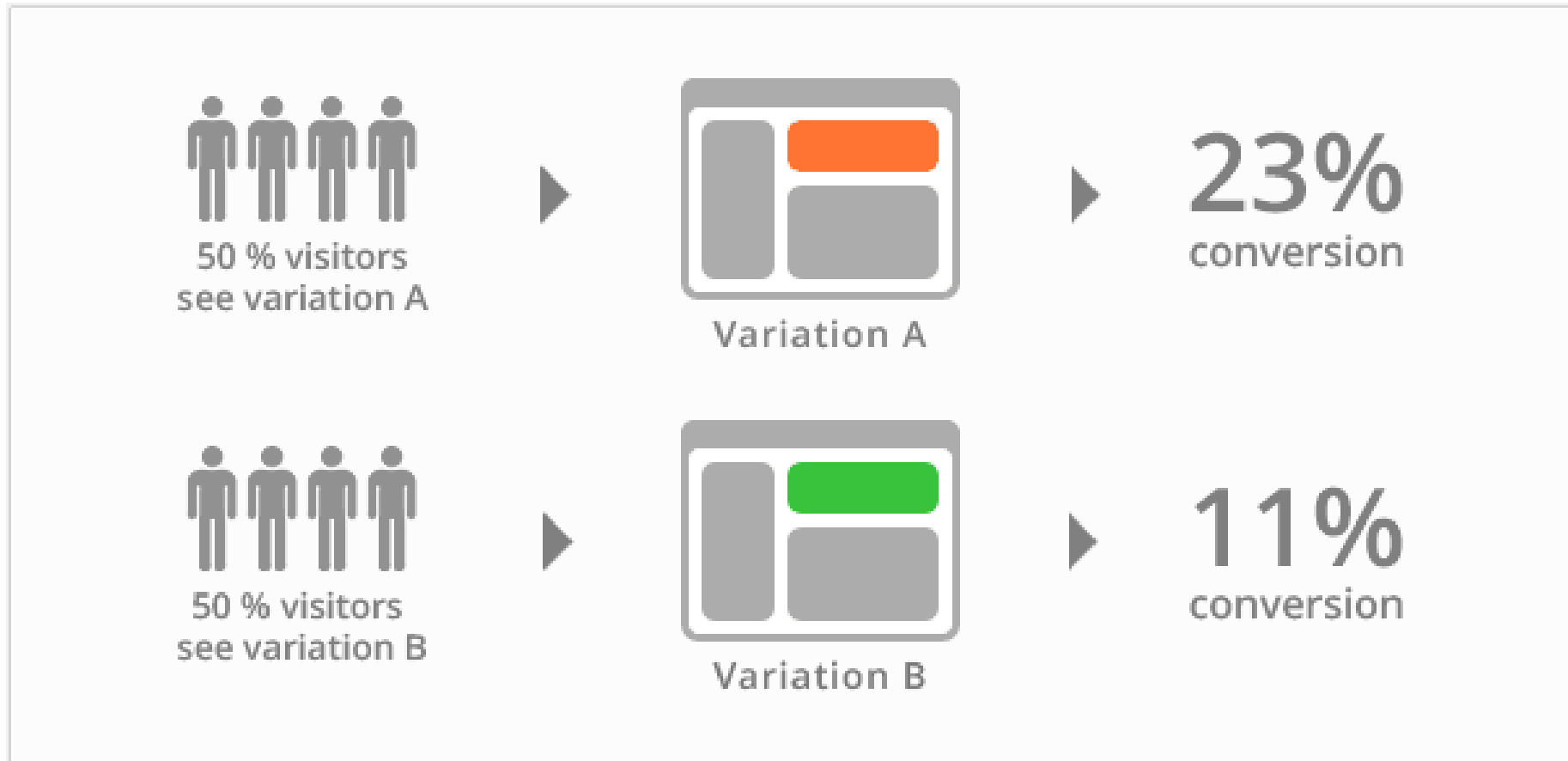
"...some companies might bake a new version for eight hours before deployment."

Every Feature is an Experiment

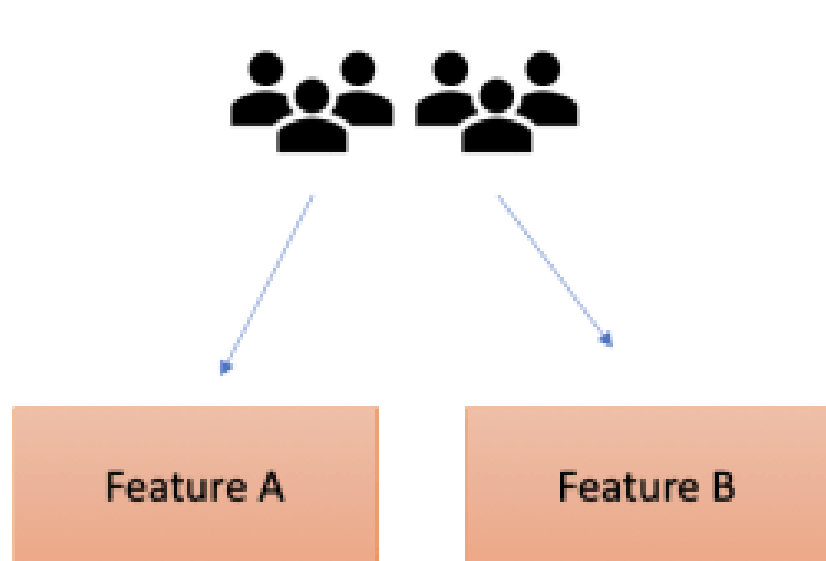
(“Top 10 Adages”)



A / B Testing

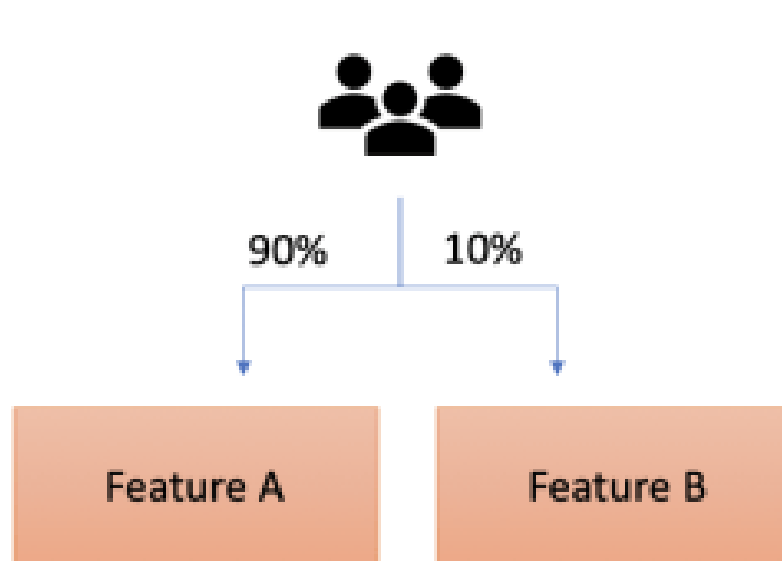


Experiments: Methods for Rollout



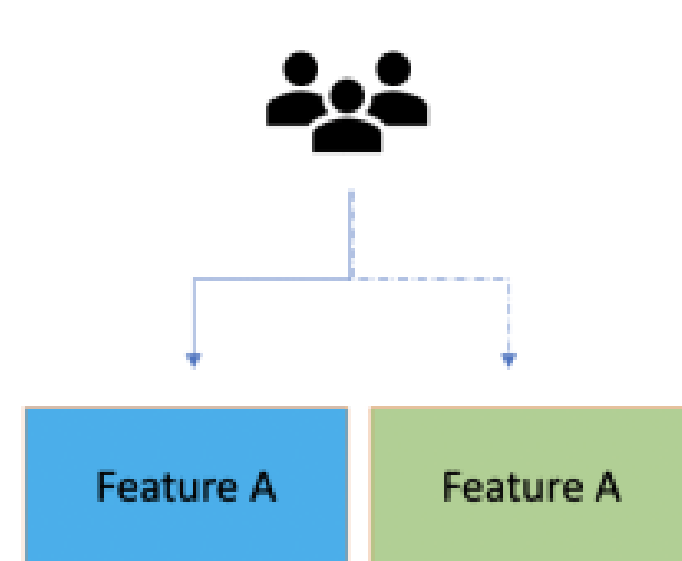
A/B Testing

Feature Flags, Networking Layer



Canary Testing

Usually Networking Layer

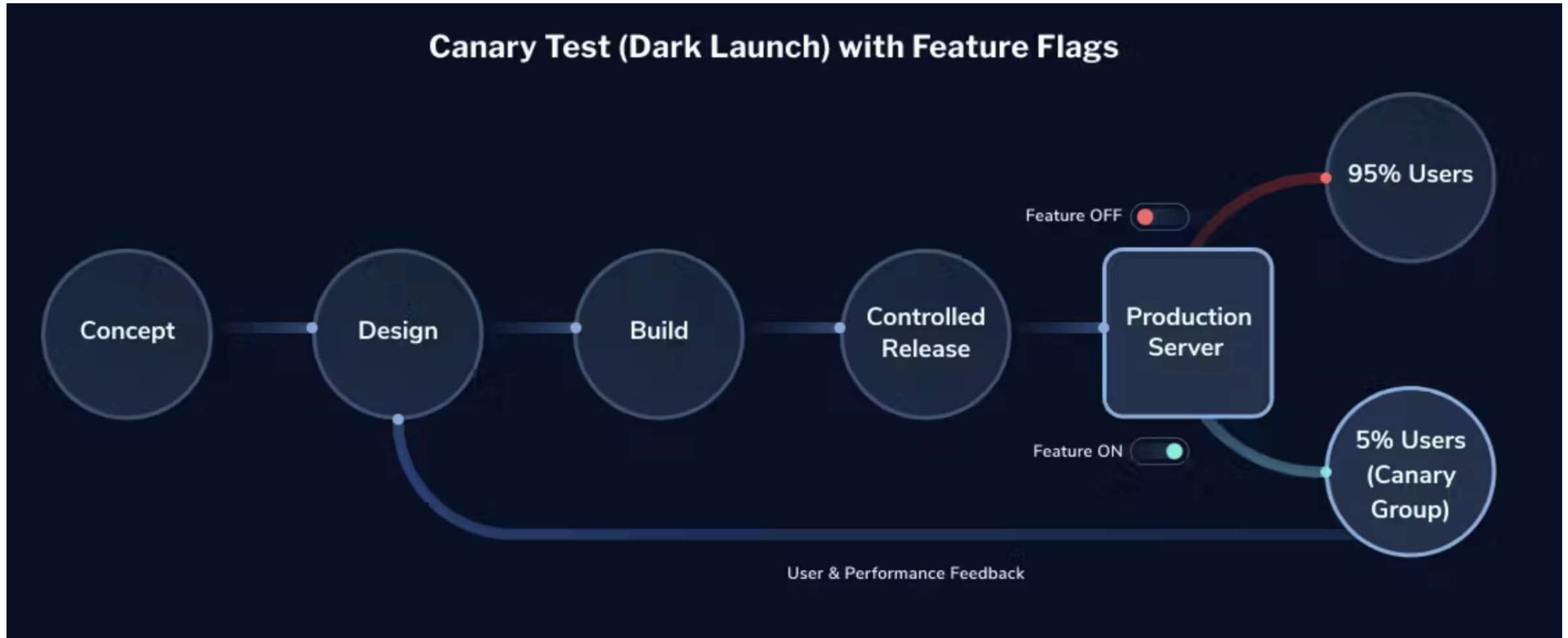


Blue Green Deployment

Separate Delivery Infrastructure

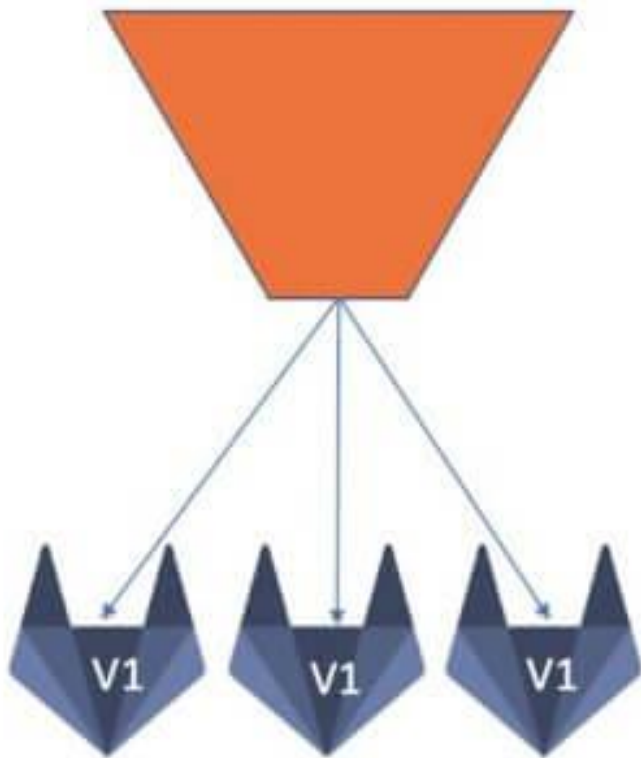
Canary Releasing

Released to real users

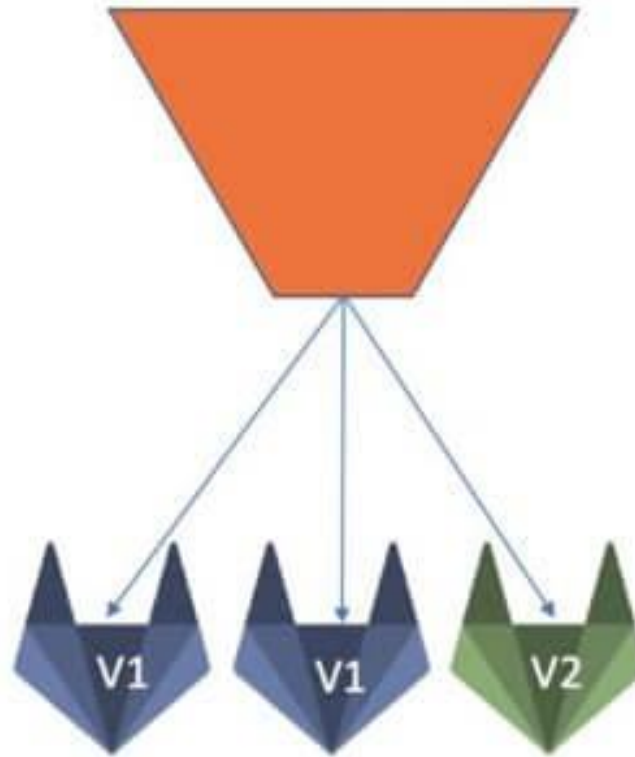


Canary Release

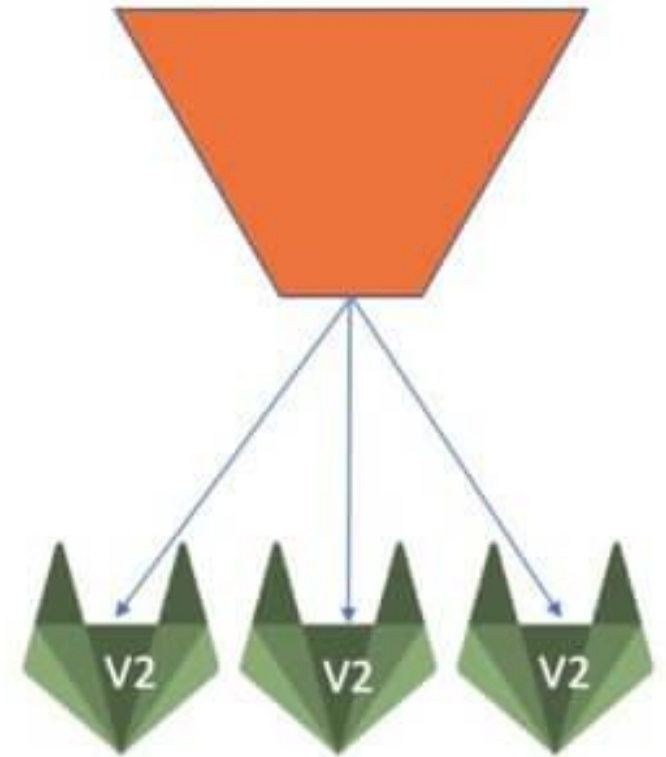
Before



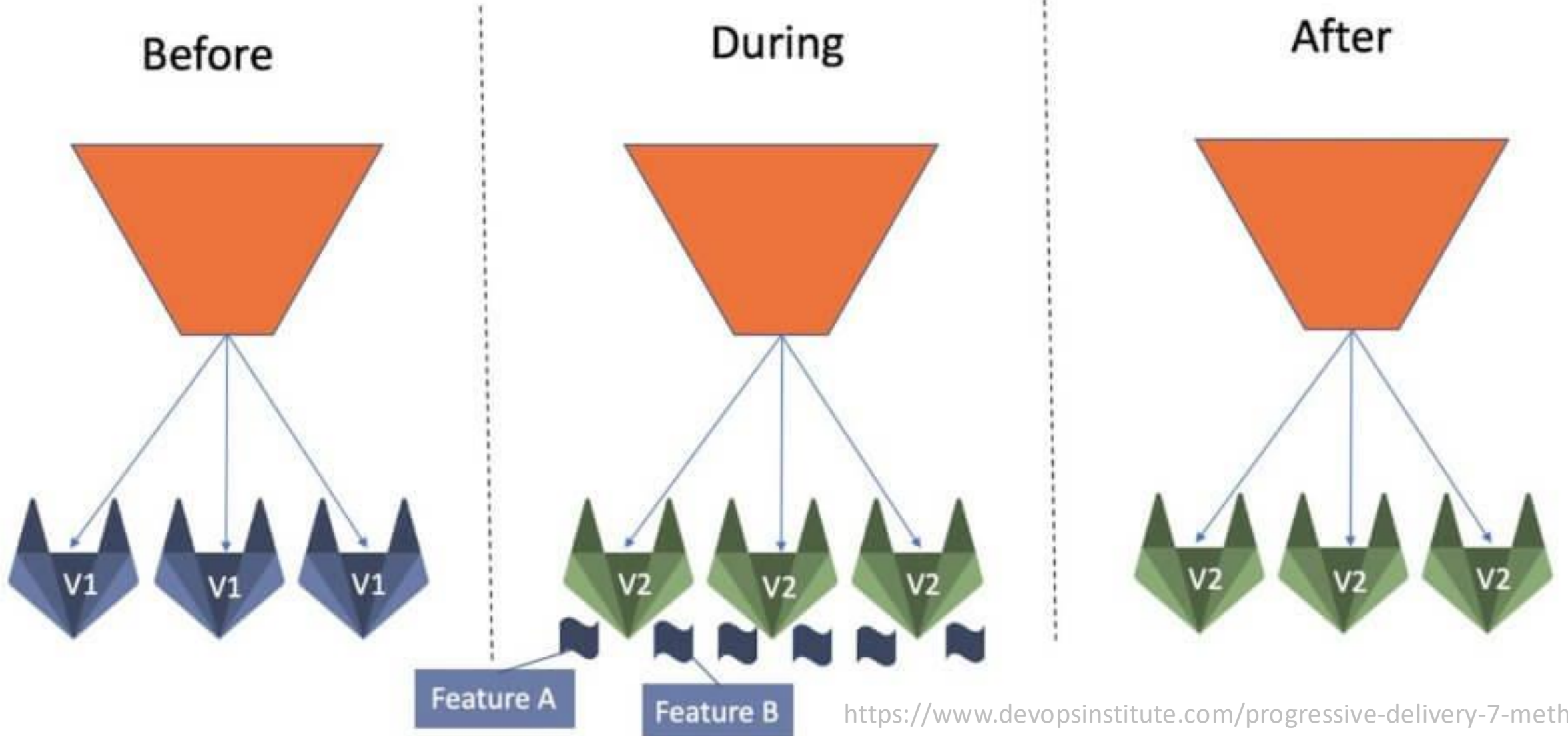
During



After

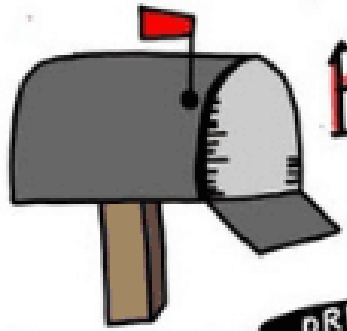


Feature Flags



Feature Flags vs Canary Release

	Feature Flags	Canary
Placement	Application	infrastructure networking layer
Orchestrated by	developers, product managers, QA	DevOps, SREs
percentage of traffic/users	V	V
Whitelisting IP addresses	V	V
Rule based (application rules - # of songs in a playlist)	V	X
Group based (domain/tier etc.)	V	X
Rollback	Granular - Specific feature	Entire Deployment



PROGRESSIVE DELIVERY

DRIT GOLOWINSKI

CONTINUOUS DELIVERY

SOFTWARE CAN BE RELEASED TO PRODUCTION AT ANY TIME

CONTINUOUS INTEGRATION

BUILD & RUN TESTS

PUSH - ENSURE IT WORKS

CI → CD

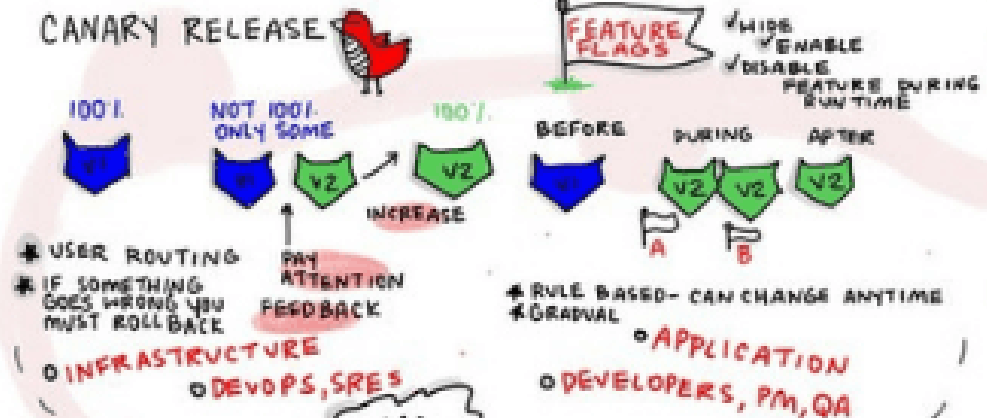
PROGRESSIVE DELIVERY

ENSURES STRENGTH

EXPERIMENTATION

... IS CONTINUOUS DELIVERY WITH FINE-GRAINED CONTROL OVER THE BLAST RADIUS
- JAMES GOVERNOR, REDMONK

CANARY RELEASE

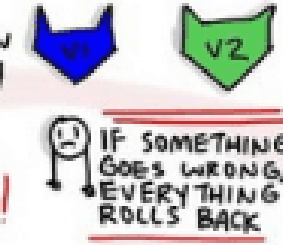


COMBINE
BEST OF BOTH WORLDS

BLUE/GREEN DEPLOYMENTS

DEPLOY TO TWO ENVIRONMENTS WHILE BLUE (EXPOSED TO PRODUCTION) DEVELOPMENT CONTINUES ON GREEN ENVIRONMENT

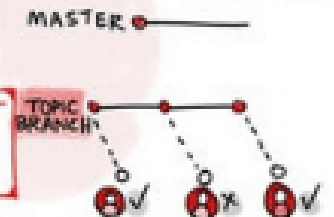
WHEN READY
SWAP!



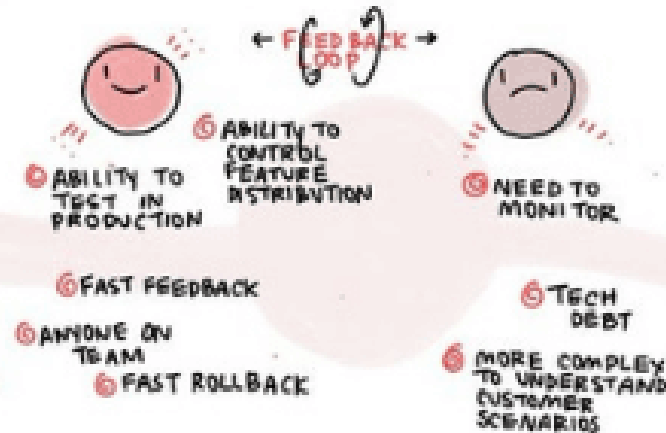
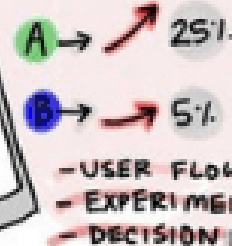
REVIEW APPS

AUTOMATED LIVE PREVIEW OF CHANGES MADE IN A FEATURE BRANCH
SPIN UP DYNAMIC ENVIRONMENT

TESTING IN PRODUCTION IS A COMMON PRACTICE



EXTENSION OF FEATURE FLAGS
MONITOR PREFERRED METHOD



Summary

- Feature Flags allow deployment without release, easy rollback. Includes some technical debt and possible bad UX experiences.
- *software features* in Continuous Deployment become *experiments*. A/B Testing, features need to demonstrate that they are useful (ROI, customer retention, engagement, conversion rates).